LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 5– RELASI ANTAR KELAS

NAMA: Anindya Sukma Dwiyanda

NIM: 103132400006

KELAS: S1SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

…

…

…

B. Guided

1. Asosiasi antara Dosen dan Mahasiswa

```
package project.guided1;

public class Mahasiswa {
    private String nim;
    private String nama;
    private int nilai;

    //Setter
    public void setNama(String nama){
        this.nama = nama;
    }
    public void setNim(String nim){
        this.nim = nim;
    }
    public void setNilai(int nilai){
        this.nilai = nilai;
    }
    //Getter
    public int getNilai(){
        return this.nilai;
    }
    public String getNama(){
        return this.nama;
    }

    public String getNim(){
        return this.nim;
    }

}
```

```
package project.guided1;

public class Dosen {
    private String kodeDosen;
    private String namaDosen;

    public void setKodeDosen(String kodeDosen){
        this.kodeDosen = kodeDosen;
```

```
    }
    public void setNamaDosen(String namaDosen){
        this.namaDosen = namaDosen;
    }
    public String getKodeDosen(){
        return this.kodeDosen;
    }
    public String getNamaDosen(){
        return this.namaDosen;
    }

    public void giveScore(Mahasiswa s, int nilai){
         s.setNilai(nilai);
    }
    public int getScore(Mahasiswa s){
         return s.getNilai();
    }

    public static void main(String[] args) {
        Mahasiswa m = new Mahasiswa();

        m.setNim("130118383");
        m.setNama("Budi");

        Dosen d = new Dosen();
        d.giveScore(m, 90);

        System.out.println("Nim :"+m.getNim());
        System.out.println("Nama :"+m.getNama());
        System.out.println("Nilai :"+d.getScore(m));
    }
}
```

Output :

```
Nim :130118383
Nama :Budi
Nilai :90
```

2. Agregasi antara Jurusan dan Mahasiswa

```
package project.guided.agregasi;

public class Mahasiswa {
    private String NIM, Nama;
```

```
    public Mahasiswa(String NIM, String Nama){
        this.NIM = NIM;
        this.Nama = Nama;
    }

    //Getter
    public String getNIM(){
        return NIM;
    }

    public String getNim(){
        return NIM;
    }

    public String getNama(){
        return Nama;
    }

    //Setter
    public void setNIM(String NIM){
        this.NIM = NIM;
    }

    public void setNim(String NIM){
        this.NIM = NIM;
    }

    public void setNama(String Nama){
        this.Nama = Nama;
    }
}
```

```
package project.guided.agregasi;

public class Jurusan {
    private static int jumlahMahasiswa;
    private String kodeJurusan, namaJurusan;
    private Mahasiswa[] daftarMahasiswa = new Mahasiswa[32];

    //contructor
    public Jurusan (String kodeJurusan, String namaJurusan){
        this.kodeJurusan = kodeJurusan;
        this.namaJurusan = namaJurusan;
    }

    //Behavior
    public void addMahasiswa(Mahasiswa m){
        this.daftarMahasiswa[jumlahMahasiswa] = m;
```

```
            jumlahMahasiswa++;
        }

        public void displayMahasiswa(){
            System.out.println("Kode Jurusan: " + this.kodeJurusan);
            System.out.println("Nama Jurusan: " + this.namaJurusan);

            System.out.println("Daftar Mahasiswa: ");

            for (int i = 0; i<jumlahMahasiswa;i++){
                System.out.println(i+1 + ". " +
                daftarMahasiswa[i].getNim() + " "
                + daftarMahasiswa[i].getNama());
            }
        }

        //Getter
        public String getKodeJurusan(){
            return kodeJurusan;
        }

        public String getNamaJurusan(){
            return namaJurusan;
        }
        //Setter

        public void setKodeJurusan(String kodeJurusan){
            this.kodeJurusan = kodeJurusan;
        }

        public void setNamaJurusan(String namaJurusan){
            this.namaJurusan = namaJurusan;
        }

        public static void main(String[] args) {
            Jurusan j = new Jurusan("D3IF", "D3 Rekayasa Perangkat Lunak
    Aplikasi");
            Mahasiswa m1 = new Mahasiswa("6701", "Budi");
            Mahasiswa m2 = new Mahasiswa("6702", "Siti");

            j.addMahasiswa(m1);
            j.addMahasiswa(m2);

            j.displayMahasiswa();
        }
}
```

Output :

```
Kode Jurusan: D3IF
Nama Jurusan: D3 Rekayasa Perangkat Lunak Aplikasi
Daftar Mahasiswa:
1. 6701 Budi
2. 6702 Siti
```

3. Komposisi Kompiter dan Komponen

```
package com.renwxyz.guided.komposisi;

public class Processor {
    private String brand;
    private int kecepatan;

    //constructor
    public Processor(String brand,int kecepatan){
        this.brand = brand;
        this.kecepatan = kecepatan;
    }

    //behaviour
    public void displaySpec(){
        System.out.println(this.brand + ", " + this.kecepatan);
    }
}
```

```
package com.renwxyz.guided.komposisi;

public class Monitor {
    private String brand;

    public Monitor(String brand){
        this.brand = brand;
    }

    public void DisplaySpec() {
        System.out.println(this.brand);
    }
}
```

```
package com.renwxyz.guided.komposisi;

public class Mouse {
    private String brand, tipe;
```

```
    public Mouse(String brand, String tipe){
        this.brand = brand;
        this.tipe = tipe;
    }

    public void displaySpec(){
        System.out.println(this.brand + ", " + this.tipe);
    }
}
```

```
package guided.komposisi;

public class Komputer {
    private Processor processor;
    private Monitor monitor;
    private Mouse mouse;

    public Komputer(String brandCpu, int kecepatanCpu, String
brandMonitor, String brandMouse, String tipeMouse){
        this.processor = new Processor(brandCpu, kecepatanCpu);
        this.monitor = new Monitor(brandMonitor);
        this.mouse = new Mouse(brandMouse, tipeMouse);
    }

    public void displaySpecKomputer() {
        System.out.println("=== SPESIFIKASI KOMPUTER ===");
        this.processor.displaySpec();
        this.monitor.DisplaySpec();
        this.mouse.displaySpec();
    }

}`
```

Output :

```
Program berhasil dikompilasi dan dijalankan:

  === SPESIFIKASI KOMPUTER ===
  Intel Core i7 3800
  ASUS
  Logitech, Wireless
```

C. Unguided

Sebuah cafe membutuhkan sistem sederhana untuk mencatat dan memproses pesanan pelanggan. Dalam sistem ini, relasi antar kelas dirancang untuk mencerminkan kondisi nyata operasional cafe.

Pengerjaan soal ini dibagi menjadi 3 bagian tugas utama yang masing-masing melatih satu jenis relasi antar kelas, diikuti dengan pengujian pada kelas utama (Main).

Tugas 1: Implementasi Komposisi (Composition)
Komposisi adalah relasi di mana sebuah objek memiliki objek lain sebagai bagian internalnya yang tidak dapat berdiri sendiri. Siklus hidup objek internal terikat penuh pada objek utamanya.
Pada sistem cafe:
Setiap item pesanan (ItemPesanan) pasti memiliki rincian harga (RincianHarga) yang mencatat harga dasar dan pajak (10%).
Objek RincianHarga harus di-instansiasi langsung di dalam constructor ItemPesanan.
Objek RincianHarga tidak pernah dibuat di main. Jika ItemPesanan dihapus, maka RincianHarga miliknya ikut hilang.
Kebutuhan Tugas 1
Buatlah dua class: RincianHarga dan ItemPesanan.
Field yang disarankan:
Class
Field
Jenis
Tipe
Keterangan
RincianHarga
hargaDasar
Instance field
double
Harga awal sebelum pajak
RincianHarga
pajak
Instance field
double
Nilai nominal pajak (10% dari harga dasar)
ItemPesanan
namaItem
Instance field
String
Nama menu (misal: "Iced Americano")
ItemPesanan
rincian
Instance field

RincianHarga
Objek internal rincian harga (Komposisi)

Method yang disarankan:
Class
Method
Return
Keterangan
RincianHarga
getHargaTotal()
double
Mengembalikan nilai hargaDasar + pajak
ItemPesanan
getNamaItem()
String
Mengembalikan nama item pesanan
ItemPesanan
getRincian()
RincianHarga
Mengembalikan objek rincian harga

Jawaban :

```
package project.unguided.komposisi;

public class ItemPesanan {
    private String namaItem;
    private RincianHarga rincian; // Objek internal (Komposisi)

    public ItemPesanan(String namaItem, double hargaDasar) {
        this.namaItem = namaItem;
        this.rincian = new RincianHarga(hargaDasar); // Di-instansiasi langsung di
constructor
    }

    public String getNamaItem() {
        return namaItem;
    }

    public RincianHarga getRincian() {
        return rincian;
    }
}
```

```
package project.unguided.komposisi;

public class RincianHarga {
    private double hargaDasar;
    private double pajak;

    public RincianHarga(double hargaDasar) {
        this.hargaDasar = hargaDasar;
        this.pajak = hargaDasar * 0.10; // Pajak 10% dari harga dasar
    }

    public double getHargaDasar() {
        return hargaDasar;
    }

    public double getPajak() {
        return pajak;
    }

    public double getHargaTotal() {
        return hargaDasar + pajak;
    }
}
```

Penjelasan :
Pada modul dijelaskan bahwa objek bagian merupakan atribut yang diinisialisasi secara langsung di dalam kelas utamanya sehingga siklus hidupnya saling terikat. Hal ini diterapkan pada kelas ItemPesanan yang menginisialisasi objek RincianHarga langsung di dalam constructor menggunakan kata kunci new.

Tugas 2: Implementasi Agregasi (Aggregation)
Agregasi adalah relasi di mana sebuah objek menampung kumpulan objek lain yang dapat berdiri sendiri. Objek yang ditampung dibuat secara independen di luar, lalu dimasukkan ke dalam objek wadah.
Pada sistem cafe:
Sebuah Pesanan menampung beberapa objek ItemPesanan menggunakan array standar.
Objek ItemPesanan dibuat terpisah terlebih dahulu (di main), lalu dimasukkan satu per satu ke dalam Pesanan melalui method tambahItem(). Jika Pesanan dibatalkan atau dihapus, objek ItemPesanan tersebut secara konsep tetap ada.
Kebutuhan Tugas 2
Buatlah class Pesanan.
Field yang disarankan:

Field
Jenis
Tipe
Keterangan
nomorMeja
Instance field
int
Nomor meja pelanggan
daftarItem
Instance field
ItemPesanan[]
Array penampung item pesanan (Agregasi)
jumlahItem
Instance field
int
Penanda jumlah item yang sudah masuk ke array

Method yang disarankan:
Method
Return
Keterangan
tambahItem(ItemPesanan item)
void
Memasukkan objek ItemPesanan dari luar ke dalam array daftarItem
tampilkanPesanan()
void
Menampilkan nomor meja dan seluruh item beserta harga total per item
getNomorMeja()
int
Mengembalikan nomor meja
getDaftarItem()
ItemPesanan[]
Mengembalikan array daftar item
getJumlahItem()
int
Mengembalikan jumlah item yang tersimpan

Jawaban :

```
package project.unguided.agregasi;
```

```
import project.unguided.komposisi.ItemPesanan;
import project.unguided.komposisi.RincianHarga;

public class Pesanan {
    private int nomorMeja;
    private ItemPesanan[] daftarItem; // Array penampung (Agregasi)
    private int jumlahItem;

    public Pesanan(int nomorMeja, int kapasitasMax) {
        this.nomorMeja = nomorMeja;
        this.daftarItem = new ItemPesanan[kapasitasMax];
        this.jumlahItem = 0;
    }

    public void tambahItem(ItemPesanan item) {
        if (jumlahItem < daftarItem.length) {
            daftarItem[jumlahItem] = item;
            jumlahItem++;
        } else {
            System.out.println("Kapasitas pesanan sudah penuh!");
        }
    }

    public void tampilkanPesanan() {
        System.out.println("=== DAFTAR PESANAN CAFE ===");
        System.out.println("Nomor Meja: " + nomorMeja);
        for (int i = 0; i < jumlahItem; i++) {
            ItemPesanan item = daftarItem[i];
            RincianHarga rin = item.getRincian();
            System.out.println((i + 1) + ". " + item.getNamaItem() + " - Rp " +
rin.getHargaTotal() +
                    " (Harga: " + rin.getHargaDasar() + ", Pajak: " + rin.getPajak()
+ ")");
        }
    }

    public int getNomorMeja() {
        return nomorMeja;
    }

    public ItemPesanan[] getDaftarItem() {
        return daftarItem;
    }

    public int getJumlahItem() {
        return jumlahItem;
    }
}
```

Penjelasan :
Berdasarkan materi modul, agregasi digambarkan melalui hubungan kelas Jurusan yang memiliki atribut array objek Mahasiswa yang dibuat secara independen. Pola ini persis diterapkan pada kelas Pesanan yang menampung sekumpulan objek ItemPesanan menggunakan array standar.

Tugas 3: Implementasi Asosiasi (Association)
Asosiasi adalah relasi interaksi antara dua objek independen yang saling menggunakan tetapi tidak saling memiliki. Objek tidak menyimpan objek lain sebagai field permanen, melainkan berinteraksi melalui parameter method.
Pada sistem cafe:
Seorang Kasir memproses pembayaran dari sebuah Pesanan.
Class Kasir tidak menyimpan objek Pesanan sebagai field, melainkan menerima objek Pesanan sebagai parameter pada method prosesPembayaran().
Kebutuhan Tugas 3
Buatlah class Kasir.
Field yang disarankan:
Field
Jenis
Tipe
Keterangan
namaKasir
Instance field
String
Nama petugas kasir

Method yang disarankan:
Method
Return
Keterangan
prosesPembayaran(Pesanan pesanan)
void
Mengakses data Pesanan, menghitung akumulasi total bayar seluruh item, lalu mencetak struk pembayaran (Asosiasi)

Jawaban :

```
package project.unguided.asosiasi;

import project.unguided.agregasi.Pesanan;
import project.unguided.komposisi.ItemPesanan;
```

```
public class Kasir {
    private String namaKasir;

    public Kasir(String namaKasir) {
        this.namaKasir = namaKasir;
    }

    public String getNamaKasir() {
        return namaKasir;
    }

    public void prosesPembayaran(Pesanan pesanan) {
        double totalSubtotal = 0;
        double totalPajak = 0;

        ItemPesanan[] items = pesanan.getDaftarItem();
        int jumlah = pesanan.getJumlahItem();

        for (int i = 0; i < jumlah; i++) {
            totalSubtotal += items[i].getRincian().getHargaDasar();
            totalPajak += items[i].getRincian().getPajak();
        }

        double totalBayar = totalSubtotal + totalPajak;

        System.out.println("=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " + this.namaKasir);
        System.out.println("Pesanan        : Meja " + pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : " + jumlah);
        System.out.println("Total Subtotal : Rp " + totalSubtotal);
        System.out.println("Total Pajak    : Rp " + totalPajak);
        System.out.println("TOTAL BAYAR    : Rp " + totalBayar);
        System.out.println("==============================");
    }
}
```

Penjelasan :
Modul mencontohkan hubungan asosiasi melalui kelas Dosen yang berinteraksi dengan kelas Mahasiswa melalui parameter method tanpa menjadikan objek tersebut sebagai atribut permanen. Pendekatan ini digunakan pada kelas Kasir yang menerima objek Pesanan sebagai parameter di dalam method pembayaran.

Tugas 4: Eksekusi dan Pengujian di Main
Buatlah class Main untuk merangkai seluruh kelas dan menguji ketiga jenis relasi.
Alur program di main
Buat 1 objek Kasir (misal: "Budi").

Buat 3 objek ItemPesanan secara independen:
"Iced Americano", harga dasar: 18000
"Beef Croissant", harga dasar: 25000
"Matcha Latte", harga dasar: 22000
Buat 1 objek Pesanan untuk Meja 4 (kapasitas array misal: 5 item).
Masukkan ketiga objek ItemPesanan ke dalam objek Pesanan menggunakan method tambahItem().
Tampilkan rincian pesanan dengan memanggil tampilkanPesanan() pada objek Pesanan.
Jalankan pemrosesan pembayaran dengan memanggil method prosesPembayaran(pesanan) dari objek Kasir.

Jawaban:

```
package project.unguided.main;

import project.unguided.agregasi.Pesanan;
import project.unguided.asosiasi.Kasir;
import project.unguided.komposisi.ItemPesanan;

public class Main {
    public static void main(String[] args) {
        // 1. Buat 1 objek Kasir
        Kasir kasir = new Kasir("Budi");

        // 2. Buat 3 objek ItemPesanan secara independen (Agregasi)
        ItemPesanan item1 = new ItemPesanan("Iced Americano", 18000);
        ItemPesanan item2 = new ItemPesanan("Beef Croissant", 25000);
        ItemPesanan item3 = new ItemPesanan("Matcha Latte", 22000);

        // 3. Buat 1 objek Pesanan untuk Meja 4 dengan kapasitas array 5
        Pesanan pesanan = new Pesanan(4, 5);

        // 4. Masukkan ketiga objek ItemPesanan ke dalam objek Pesanan
        pesanan.tambahItem(item1);
        pesanan.tambahItem(item2);
        pesanan.tambahItem(item3);

        // 5. Tampilkan rincian pesanan
        pesanan.tampilkanPesanan();
        System.out.println();

        // 6. Jalankan pemrosesan pembayaran oleh Kasir (Asosiasi)
        kasir.prosesPembayaran(pesanan);
    }
```

```
    }
```

Output:

```
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
==============================
```

D. Kesimpulan

Berdasarkan praktikum dan penyelesaian tugas unguided modul lima ini, dapat ditarik beberapa kesimpulan penting terkait penerapan konsep pemrograman berorientasi objek. Relasi komposisi berhasil diimplementasikan dengan baik antara kelas item pesanan dan rincian harga, di mana objek rincian harga diciptakan secara internal di dalam konstruktor item pesanan sehingga kelangsungan hidupnya bergantung penuh pada objek utamanya. Selain itu, relasi agregasi berhasil diterapkan pada kelas pesanan yang menampung sekumpulan objek item pesanan di dalam array standar, di mana objek item pesanan bersifat independen dan dapat diciptakan terlebih dahulu sebelum dimasukkan ke dalam wadah pesanan. Relasi asosiasi juga berhasil dipraktikkan pada hubungan antara kelas kasir dan pesanan, yang memperlihatkan bahwa interaksi terjadi hanya saat pemanggilan metode pembayaran melalui pengiriman parameter tanpa harus menyimpan objek pesanan sebagai atribut permanen di dalam kelas kasir. Keseluruhan perancangan program ini didukung oleh penerapan prinsip enkapsulasi melalui penggunaan field privat beserta metode getter dan setter, serta kepatuhan terhadap konvensi penamaan standar Java yang membantu menjaga kebersihan struktur kode serta keamanan data di dalam objek.